Український католицький університет

Statistics and Econometrics

Завдання 4.

Регуляризація, дерева та логістична регресія

Задача 7

Виконав: **Владислав Григола**

2026

Зміст

1. Lasso та Ridge **3**
1.а Концепція Lasso . . . 3
1.б Масштабування та кодування . . . 4
1.в Вибір штрафу Lasso . . . 6
1.г Ridge та OLS . . . 8

2. Дерева та ансамблі **10**
2.а Критерій поділу . . . 10
2.в Порядкові та категоріальні ознаки . . . 11
2.г Обмеження розміру CART . . . 12
2.д Обрізання дерева . . . 14
2.е Покращення в корені . . . 15
2.ж Важливості дерева та коефіцієнти . . . 16
2.і Помилка ансамблів . . . 18
2.к Час та важливості моделей . . . 20

3. Логістична регресія **22**
3.а Відбір за AIC . . . 23
3.б Відношення шансів . . . 26
3.в Значущість і знаки . . . 26
3.г П’ять прогнозів . . . 28
3.д Матриця помилок при 0.5 . . . 30
3.е ROC та поріг Юдена . . . 31
3.ж Класифікація з новим порогом . . . 32

Спільні припущення та підготовка

Опціональні пункти 2.б, 2.з, 2.и пропущено.

Дані Ames Housing: 2930 продажів, SalePrice у доларах США, площі у квадратних футах. Випадкове розбиття 80/20 із seed 42 дає 2344 навчальних і 586 відкладених рядків, однакових для всіх частин; у частині 2 відкладені рядки називаємо валідаційними, бо на них порівнюємо кілька моделей. Рівень значущості 0.05.

У частинах 1 і 2 вилучаємо Order, PID і GarageYrBlt; MSSubClass і текстові ознаки кодуємо індикаторами з одним базовим рівнем, структурне NA є окремою категорією, числові пропуски заповнюємо навчальною медіаною, числові ознаки стандартизуємо. Усі перетворення навчаються лише на навчальній частині, штрафи вибираємо 5-блочною крос-валідацією. У частині 3 клас 1 означає SalePrice понад 130000 доларів; використовуємо лише числові ознаки, їх підготовку показано на початку частини.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
df = pd.read_csv(DATA_PATH, sep="\t", keep_default_na=False, na_values=[""])
df.columns = df.columns.str.replace(" ", "").str.replace("/", "")
for column in df.select_dtypes(exclude="number").columns:
    df[column] = df[column].str.strip()
df["MSSubClass"] = df["MSSubClass"].astype(str)
X = df.drop(columns=["SalePrice", "Order", "PID", "GarageYrBlt"])
y = df["SalePrice"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED
)
numeric_columns = X.select_dtypes(include="number").columns.tolist()
categorical_columns = X.select_dtypes(exclude="number").columns.tolist()
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

	Значення
Спостережень	2930
Навчальна частина	2344
Перевірочна частина	586
Числових ознак	34
Категоріальних ознак	44
Пропусків у SalePrice	0

FireplaceQu	Кількість
NA	1422
Gd	744
TA	600
Fa	75
Po	46
Ex	43

1. Lasso та Ridge

Регресія ласо (Lasso) та рідж-регресія (Ridge) є методами регуляризації, які використовуються для стабілізації оцінювання параметрів.

1.a Концепція Lasso

Умова завдання

Поясніть своїми словами концепцію регресії ласо. Опишіть сценарій, за якого звичайна лінійна регресія зазнає невдачі (наприклад, через мультиколінеарність або коли $p > n$), але регресію ласо все ще можна оцінити. Чому нам потрібно максимізувати цільову функцію чисельно (на відміну від рідж-регресії, яка має аналітичний розв'язок)?

Lasso додає до суми квадратів залишків штраф за суму модулів коефіцієнтів. Штраф зменшує коефіцієнти, а частину може зробити точно нульовими, тому Lasso одночасно виконує відбір ознак.

$$RSS + \lambda \sum_{j=1}^{p} |\beta_j| \longrightarrow \min .$$

Тут RSS позначає суму квадратів залишків, $\lambda > 0$ є силою штрафу, p кількістю ознак, а β_j коефіцієнтом ознаки з номером j. Вільний член у цьому записі опущено завдяки центруванню даних.

Якщо дві ознаки дублюють одна одну або ознак більше, ніж спостережень ($p > n$, де n позначає кількість спостережень), матриця $X^T X$ вироджена і звичайна формула МНК з оберненою матрицею не працює. Тут X позначає матрицю ознак. Мінімум RSS усе ще існує, але коефіцієнти можуть бути неєдиними. Сильна, але неточна колінеарність робить оцінки нестійкими. Lasso зі штрафом можна оцінити й у цих випадках, хоча при точному дублюванні ознак його коефіцієнти теж не обов'язково єдині.

Функцію потрібно мінімізувати, а не максимізувати. Модуль не має похідної в нулі, тому для загальної матриці ознак немає простої матричної формули, як у Ridge. Один із чисельних методів, покоординатний спуск, по черзі оновлює коефіцієнти до збіжності. Для спеціальних випадків, наприклад ортогональних ознак, явний розв'язок існує.

1.б Масштабування та кодування

Умова завдання

Для звичайної регресії МНК змінні рідко нормуються. Однак у випадку ласо-регресії масштабування є критичним, оскільки штраф накладається безпосередньо на абсолютну величину коефіцієнтів. Чому це так? Масштабуйте числові дані за допомогою StandardScaler. Чи можна / чи потрібно масштабувати бінарні змінні (даммі) в такий самий спосіб? Як обробити факторні змінні (наприклад, FireplaceQu) через OneHotEncoder?

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Кожне перетворення навчається лише на переданій навчальній частині.
numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="constant", fill_value="Missing")),
    ("encode", OneHotEncoder(drop="first", handle_unknown="ignore", sparse_output=False)),
])
preprocessor = ColumnTransformer([
    ("num", numeric_pipe, numeric_columns),
    ("cat", categorical_pipe, categorical_columns),
], verbose_feature_names_out=False)
prepared = clone(preprocessor).fit(X_train)
Z_train = prepared.transform(X_train)
Z_test = prepared.transform(X_test)
feature_names = prepared.get_feature_names_out()
scaler = prepared.named_transformers_["num"].named_steps["scale"]
encoder = prepared.named_transformers_["cat"].named_steps["encode"]
fireplace_index = categorical_columns.index("FireplaceQu")
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Ознака	Середнє після заповнення	Масштаб StandardScaler
LotFrontage	69.0051	21.3641
LotArea	10127.9	8049.19
OverallQual	6.06442	1.38822
OverallCond	5.58191	1.10542
YearBuilt	1970.51	30.335
YearRemodAdd	1983.92	20.7819
MasVnrArea	97.741	171.73
BsmtFinSF1	442.503	452.032
BsmtFinSF2	50.0759	170.327
BsmtUnfSF	554.354	433.607
TotalBsmtSF	1047	436.383
1stFlrSF	1154.81	385.032
2ndFlrSF	333.688	427.05
LowQualFinSF	4.35708	44.3145
GrLivArea	1492.86	504.512
BsmtFullBath	0.431314	0.525356
BsmtHalfBath	0.0593003	0.241544
FullBath	1.55717	0.552044
HalfBath	0.370734	0.499502
BedroomAbvGr	2.84514	0.820177
KitchenAbvGr	1.04309	0.211294
TotRmsAbvGrd	6.40913	1.56332
Fireplaces	0.598976	0.652848
GarageCars	1.74701	0.746145
GarageArea	469.082	212.342
WoodDeckSF	94.1131	124.825
OpenPorchSF	47.3417	68.0229
EnclosedPorch	24.7317	67.0166

Ознака	Середнє після заповнення	Масштаб StandardScaler
3SsnPorch	2.42278	24.5191
ScreenPorch	16.0102	55.809
PoolArea	2.58276	38.316
MiscVal	58.055	623.242
MoSold	6.18473	2.70783
YrSold	2007.79	1.31484

Скорочено: пропущено колонки «Середнє після стандартизації» (усі приблизно 0) та «SD після стандартизації» (усі дорівнюють 1).

```
Рівні FireplaceQu: ['Ex' 'Fa' 'Gd' 'NA' 'Po' 'TA']
Базовий рівень: Ex
Даммі FireplaceQu: ['FireplaceQu_Fa', 'FireplaceQu_Gd', 'FireplaceQu_NA', 'FireplaceQu_Po', 'FireplaceQu_TA']
Розмір матриці після кодування: (2344, 294)
```

Ознака	Невідомих значень у перевірочній частині
Exterior2nd	1
Electrical	1
KitchenQual	1
MiscFeature	1

Результати та інтерпретація

Якщо площу перевести у більші одиниці, її коефіцієнт має збільшитися, щоб прогноз залишився тим самим. RSS від цього не зміниться, а штраф за модуль коефіцієнта зросте. Тому без масштабування Lasso нерівномірно штрафує однакові за змістом ефекти.

`StandardScaler` обчислює $z_{ij} = (x_{ij} - \bar{x}_j)/s_j$, де x_{ij} є значенням ознаки j для будинку i, $\bar{x}_j$ її навчальним середнім, а s_j навчальним стандартним відхиленням із дільником n; z_{ij} є стандартизованим значенням. Після цього числовий коефіцієнт описує зміну ціни при збільшенні ознаки на одне навчальне стандартне відхилення. Для звичайного МНК з вільним членом таке оборотне перетворення не змінює підгонку, хоча може покращити числову стійкість.

Даммі теж можна стандартизувати: для частки одиниць q її стандартне відхилення дорівнює $\sqrt{q(1-q)}$. Це змінює відносну силу штрафу для рідкісних і частих категорій. Єдиного обов'язкового вибору немає. Тут залишаємо даммі у шкалі 0/1, щоб коефіцієнт відповідав переходу від базової категорії до заданої; це свідомий вибір шкали штрафу.

`OneHotEncoder(drop="first")` створює індикатори для всіх категорій, крім базової. Для `FireplaceQu` базовий рівень `Ex`, а `NA` означає відсутність каміна й має окрему даммі.

Параметри масштабування та перелік категорій визначаємо за навчальними даними.

1.в Вибір штрафу Lasso

Умова завдання

Запустіть регресію ласо за допомогою Lasso або LassoCV з коефіцієнтом регуляризації $\alpha > 0$ (α в sklearn відповідає λ в теоретичних формулах). Побудуйте графік залежності оцінених параметрів від α. Яке значення α ви б рекомендували? Визначте оптимальний α за допомогою 5-блочної крос-валідації (LassoCV).

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
lasso_alphas = np.logspace(1, 4, 35)
# LassoCV виконує п'ять блоків на вже підготовленій матриці.
lasso_cv = LassoCV(alphas=lasso_alphas, cv=cv, max_iter=50000, tol=1e-6)
lasso_cv.fit(Z_train, y_train)
# Перевіряємо вибір штрафу без спільного масштабування між блоками.
lasso_search = GridSearchCV(
    Pipeline([("prepare", clone(preprocessor)),
              ("model", Lasso(max_iter=50000, tol=1e-6))]),
    {"model__alpha": lasso_alphas}, cv=cv,
    scoring="neg_mean_squared_error", n_jobs=1,
)
lasso_search.fit(X_train, y_train)
lasso_model = lasso_search.best_estimator_.named_steps["model"]
lasso_alpha = lasso_search.best_params_["model__alpha"]
# Центрування потрібне, бо lasso_path не оцінює вільний член.
path_alphas, coefficients, gaps = lasso_path(
    Z_train - Z_train.mean(axis=0), y_train.to_numpy() - y_train.mean(),
    alphas=np.logspace(1, 5, 60), max_iter=50000, tol=1e-6,
)
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Метод	alpha	CV MSE, дол.^2
LassoCV: спільна підготовка	76.2699	9.81199e+08
Pipeline: підготовка в кожному блоці	76.2699	9.82068e+08

```
Ненульових коефіцієнтів: 126 із 294
Ітерацій фінальної моделі: 579
Відносний dual gap: 9.830565287744918e-07
```

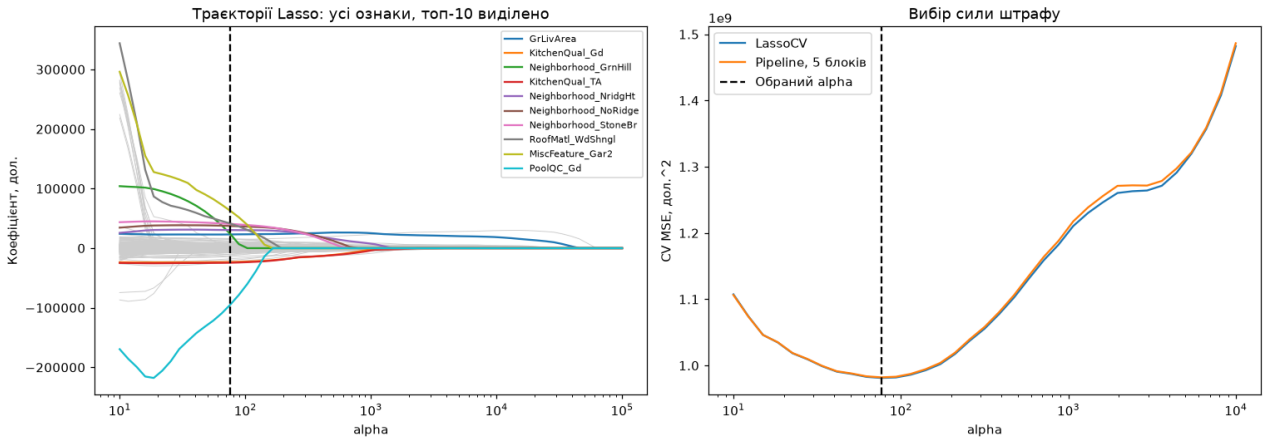


Рис. 1: Траєкторії коефіцієнтів і вибір штрафу Lasso.

	Коефіцієнт Lasso
PoolQC_Gd	-94452.8
MiscFeature_Gar2	62519.3
RoofMatl_WdShngl	41280.6
Neighborhood_StoneBr	40662.6
Neighborhood_NoRidge	37231.7
Neighborhood_NridgHt	30428.6
KitchenQual_TA	-24421.7
Neighborhood_GrnHill	23490.9
KitchenQual_Gd	-23299.2
GrLivArea	23057.6
BldgType_Twnhs	-22854.1
KitchenQual_Fa	-20387.4
BldgType_TwnhsE	-17866
LandContour_HLS	15980.7
Neighborhood_Somerst	15094.3
Exterior1st_BrkFace	15034.8
BsmtQual_Gd	-14333.7
OverallQual	14157.1
LotShape_IR3	-12279
Condition1_PosN	11470.3

Скорочено: із 126 ненульових коефіцієнтів Lasso показано 20 найбільших за модулем.

Результати та інтерпретація

Середня квадратична помилка прогнозу визначається як $MSE = \frac{1}{m}\sum_{i=1}^{m}(y_i - \widehat{y}_i)^2$, де m є кількістю перевірочних спостережень, y_i фактичною ціною, а $\widehat{y}_i$ прогнозом для будинку i. Одиниці MSE тут дол.^{2}. Для крос-валідації обчислюємо цю помилку окремо на кожному відкладеному блоці.

Для кожного штрафу навчаємо модель на чотирьох блоках і прогнозуємо п'ятий. Вибираємо мінімум $CV(\alpha) = \frac{1}{5}\sum_{k=1}^{5} MSE_k(\alpha)$, де MSE_k є середньою квадратичною помилкою на відкладеному блоці k. Це найкраще значення серед перевіреної сітки, а не доведений мінімум серед усіх додатних чисел.

`LassoCV` виконує вимогу завдання, але отримує вже масштабовану матрицю: середні та медіани тоді використовують усю навчальну частину, включно з внутрішніми перевірочними блоками. Тому додатково застосовано `GridSearchCV` до повного `Pipeline`, де підготовка відбувається окремо в кожному блоці. Для фінальної моделі рекомендую штраф із цього другого порівняння; конкретні значення, CV MSE та кількість ненульових коефіцієнтів наведено вище.

На графіку зростання штрафу приводить коефіцієнти до нуля; окремі траєкторії можуть бути немонотонними через зв'язки між ознаками. Вільний член зберігається. Нульовий коефіцієнт означає, що ознака не використовується цією моделлю за обраного штрафу; це не доказ відсутності її економічного впливу.

В обох способах вибору отримано $\alpha \approx 76.27$. Це внутрішня точка сітки, а не її межа. У sklearn цільова функція ділить RSS на $2n$, тому α бібліотеки відповідає $\lambda/(2n)$ у записі формули з пункту 1.a: для порівняння з нею $\lambda = 2n\alpha$. Lasso лишає ненульовими 126 коефіцієнтів із 294. За дуже великих штрафів модель втрачає корисні ознаки, що видно зі зростання CV MSE.

1.г Ridge та OLS

Умова завдання

Розгляньте рідж-регресію (Ridge або RidgeCV). Оцініть її для оптимального значення α, знайденого через крос-валідацію. Порівняйте оцінені параметри та їхні дисперсії для рідж-регресії та множинної лінійної регресії (LinearRegression). Чого ви очікуєте від такого порівняння з точки зору компромісу між зміщенням та дисперсією (bias-variance tradeoff)? Не забудьте підігнати лінійну регресію до попередньо масштабованих змінних.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Прогноз перед обчисленням: Ridge зменшить умовні дисперсії коефіцієнтів,
# але його помилка на нових даних не обов'язково буде меншою за МНК.
print("Прогноз: менші умовні дисперсії Ridge; перевага за перевірочною MSE не гарантована.")
ridge_alphas = np.logspace(-2, 5, 40)
ridge_search = GridSearchCV(
    Pipeline([("prepare", clone(preprocessor)), ("model", Ridge(solver="svd"))]),
    {"model__alpha": ridge_alphas}, cv=cv,
    scoring="neg_mean_squared_error", n_jobs=1,
)
ridge_search.fit(X_train, y_train)
ridge_model = ridge_search.best_estimator_.named_steps["model"]
ridge_alpha = ridge_search.best_params_["model__alpha"]
ols_model = LinearRegression().fit(Z_train, y_train)
# Усі три моделі використовують ту саму матрицю й ті самі рядки.
metrics = []
for name, model in [("OLS", ols_model), ("Lasso", lasso_model), ("Ridge", ridge_model)]:
    metrics.append({
        "Модель": name,
        "alpha": {"OLS": 0, "Lasso": lasso_alpha, "Ridge": ridge_alpha}[name],
        "Навчальна MSE, дол.^2": mean_squared_error(y_train, model.predict(Z_train)),
        "Перевірочна MSE, дол.^2": mean_squared_error(y_test, model.predict(Z_test)),
        "Норма коефіцієнтів": np.linalg.norm(model.coef_),
    })
# За точної колінеарності МНК повертає розв'язок найменшої норми.
# Псевдообернена матриця дозволяє порівняти саме цей розв'язок із Ridge.
Z_centered = Z_train - Z_train.mean(axis=0)
rank = ols_model.rank_
sigma2 = np.sum((y_train - ols_model.predict(Z_train)) ** 2) / (len(y_train) - rank - 1)
Z_pinv = np.linalg.pinv(Z_centered, rcond=max(Z_centered.shape) * np.finfo(float).eps)
A_ridge = np.linalg.solve(
    Z_centered.T @ Z_centered + ridge_alpha * np.eye(Z_train.shape[1]),
    Z_centered.T,
)
var_ols = sigma2 * np.sum(Z_pinv ** 2, axis=1)
var_ridge = sigma2 * np.sum(A_ridge ** 2, axis=1)
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Прогноз: менші умовні дисперсії Ridge; перевага за перевірочною MSE не гарантована.
```

Модель	alpha	Навчальна MSE, дол.^2	Перевірочна MSE, дол.^2	Норма коефіцієнтів
OLS	0	3.50384e+08	1.24388e+09	2.21385e+06
Lasso	76.2699	5.5878e+08	8.54966e+08	158985
Ridge	2.15443	4.95726e+08	8.43583e+08	206753

```
Ridge: мінімальна CV MSE, дол.^2: 976187864.1906097
Ранг центрованої матриці / кількість ознак: 280 / 294
Оцінка дисперсії похибки, дол.^2: 398109338.6626817
Сума дисперсій OLS та Ridge: 61977050639.282486 4795464375.7660475
```

Ознака	OLS	Ridge	Var OLS, дол.^2	Var Ridge, дол.^2	Var Ridge / Var OLS
LotFrontage	1350.95	-1642.45	439737	380368	0.86499
LotArea	5462.63	4115.2	422033	343163	0.81312
OverallQual	8711.49	11650.9	912309	804783	0.882139
OverallCond	6698.35	5865.8	420234	378228	0.900042
YearBuilt	12549.5	8961.85	2.96605e+06	2.40766e+06	0.811738
YearRemodAdd	1021.81	1292.28	581260	541923	0.932324
MasVnrArea	3286.45	2327.17	553265	495694	0.895942
BsmtFinSF1	9812.54	3912.92	2.90127e+06	501621	0.172897
BsmtFinSF2	2459.41	2057.8	967681	570356	0.589405
BsmtUnfSF	744.428	-222.278	2.69546e+06	421994	0.156557
TotalBsmtSF	9661.19	3150.53	3.71671e+06	751948	0.202315
1stFlrSF	6470.09	6119.32	1.06513e+06	857241	0.804824
2ndFlrSF	13632.9	12585.4	1.19029e+06	965300	0.810979
LowQualFinSF	-550.222	-16.5816	279711	236961	0.847162
GrLivArea	16429.2	15321.8	682862	577149	0.845191
BsmtFullBath	460.495	2122.99	464571	433304	0.932696
BsmtHalfBath	-190.142	-213.161	233474	220651	0.945079
FullBath	1406.64	2804.55	652484	599414	0.918664
HalfBath	782.596	1479.16	503797	464521	0.92204
BedroomAbvGr	-2980.8	-1342.17	580385	523818	0.902536
KitchenAbvGr	-1851.62	-2065.97	809936	649437	0.801837
TotRmsAbvGrd	-17.3626	41.1112	965697	883210	0.914582
Fireplaces	3757.38	4051.23	1.22215e+06	1.10581e+06	0.904806
GarageCars	3417.26	7005.77	1.35094e+06	1.21714e+06	0.900956
GarageArea	2437.46	56.6066	1.18335e+06	1.08765e+06	0.919122
WoodDeckSF	1651.1	1517.17	254233	240649	0.946566
OpenPorchSF	-8.12204	-450.027	257176	239440	0.931036
EnclosedPorch	-81.8997	271.508	267212	243946	0.912931
3SsnPorch	363.573	720.703	194647	185448	0.95274
ScreenPorch	2720.29	3463	215695	203069	0.941465
PoolArea	-628.622	2435.42	4.18914e+06	415538	0.0991941
MiscVal	211.243	-7528.77	1.09159e+06	322062	0.29504
MoSold	-238.493	-214.678	200895	191388	0.952681
YrSold	-1199.57	-1347.68	210233	200782	0.955045

Скорочено: показано 34 числові ознаки; 260 рядків індикаторів категорій пропущено, їхні дисперсії входять у суми вище та в рисунок нижче.

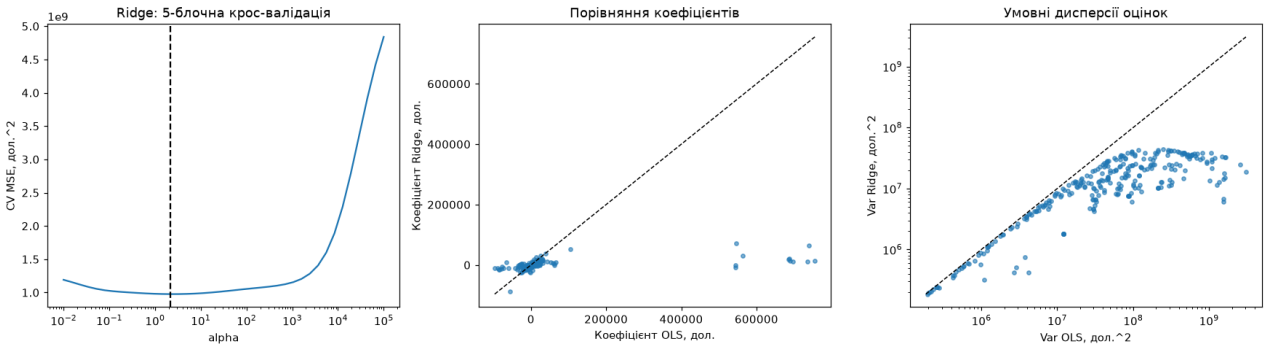


Рис. 2: Крос-валідація Ridge, коефіцієнти та умовні дисперсії.

Результати та інтерпретація

Очікування до обчислень: додатний штраф Ridge зменшить умовні дисперсії оцінок, але внесе зміщення. Менша дисперсія не гарантує меншої помилки прогнозу на нових даних.

Обидві моделі оцінюємо на тих самих підготовлених ознаках. Ранг центрованої матриці становить 280 за 294 ознак, тому коефіцієнти МНК не єдині. Порівнюємо Ridge з розв’язком МНК, який повертає `LinearRegression`: серед розв’язків він має найменшу суму квадратів коефіцієнтів. Для нього використовуємо псевдообернену матрицю, яка працює і за залежних стовпців.

Фіксуємо матрицю ознак і вибраний штраф та припускаємо, що похибки некорельовані й мають однакову дисперсію. Оцінки матриць коваріацій коефіцієнтів мають вигляд

$$\widehat{\mathrm{Var}}(\widehat{\beta}_{OLS}) = \widehat{\sigma}^2 Z^{+}(Z^{+})^T, \qquad \widehat{\mathrm{Var}}(\widehat{\beta}_R) = \widehat{\sigma}^2 A_R A_R^T, \qquad A_R = (Z^T Z + \alpha I)^{-1} Z^T.$$

Тут $\widehat{\beta}_{OLS}$ і $\widehat{\beta}_R$ є оцінками коефіцієнтів МНК і Ridge, Z є центрованою матрицею ознак, а Z^{+} її псевдообернеою матрицею. Матриця A_R перетворює центровані ціни на оцінки Ridge, α є силою штрафу, I одиничною матрицею, T позначає транспонування, а -1 обернення матриці. Оцінку дисперсії похибок $\widehat{\sigma}^2$ отримуємо, ділячи суму квадратів залишків МНК на $n-r-1$, де n є кількістю навчальних спостережень, r рангом Z, а одиниця враховує вільний член. Діагоналі матриць дають дисперсії окремих коефіцієнтів у таблиці.

Результати після обчислень

Крос-валідація вибрала для Ridge $\alpha \approx 2.154$. Коефіцієнт `LotArea` зменшився з 5463 до 4115 дол. на одне навчальне стандартне відхилення площі. Сума умовних дисперсій зменшилася з $6.198 \cdot 10^{10}$ до $4.795 \cdot 10^{9}$ дол.², як і очікували. Зміщення коефіцієнтів окремо не оцінювали.

2. Дерева та ансамблі

На наступному кроці ми моделюємо SalePrice за допомогою дерев регресії, використовуючи DecisionTreeRegressor з бібліотеки sklearn.tree.

2.a Критерій поділу

Умова завдання

Нехай GrLivArea буде першою змінною, яка використовується для розщеплення (splitting). Запишіть відповідну задачу оптимізації (мінімізацію суми квадратів залишків RSS для двох дочірніх вузлів) та поясніть, як працює алгоритм жадібного пошуку точки розщеплення.

Для першого розщеплення фіксуємо $x_i = GrLivArea_i$ і розв'язуємо

$$\min_s \left[\sum_{i:x_i \le s} (y_i - \bar{y}_L(s))^2 + \sum_{i:x_i > s} (y_i - \bar{y}_R(s))^2 \right].$$

Тут x_i є житловою площею будинку i, y_i його ціною, s порогом, а $\bar{y}_L(s)$ та $\bar{y}_R(s)$ середніми цінами лівої та правої груп. Обидві групи мають бути непорожніми. Для фіксованого порогу саме середні мінімізують суму квадратів у групах.

Алгоритм сортує різні значення площі та перевіряє пороги посередині між сусідніми різними значеннями. Розщеплення між однаковими значеннями не допускається. Обираємо поріг із найменшим сумарним RSS дочірніх вузлів. Пошук жадібний: він оптимізує лише поточний поділ, не перебираючи всі можливі майбутні дерева.

У цьому пункті ознаку задано умовою. У повній моделі нижче CART сам вибирає і ознаку, і поріг; її кореневий поділ не зобов'язаний використовувати `GrLivArea`.

2.в Порядкові та категоріальні ознаки

Умова завдання

Тепер розгляньте змінну OverallQual. Вона приймає 10 різних упорядкованих значень (від 1 до 10). Як алгоритм дерева на Python визначає оптимальне розщеплення для такого типу дискретних або рангових змінних? Опишіть логіку обробки категоріальних та порядкових ознак у деревах.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Окреме дерево з одним поділом демонструє роботу з порядковою ознакою.
quality_tree = DecisionTreeRegressor(max_depth=1, random_state=SEED)
quality_tree.fit(X_train[["OverallQual"]], y_train)
quality_values = np.sort(X_train["OverallQual"].unique())
print("Спостережені рівні:", quality_values)
print("Кандидати між сусідніми рівнями:", (quality_values[:-1] + quality_values[1:]) / 2)
print("Обраний поріг OverallQual:", quality_tree.tree_.threshold[0])
print("Середні ціни в лівому та правому листках:", quality_tree.tree_.value[1:, 0, 0])
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Спостережені рівні: [ 1  2  3  4  5  6  7  8  9 10]
Кандидати між сусідніми рівнями: [1.5 2.5 3.5 4.5 5.5 6.5 7.5 8.5 9.5]
Обраний поріг OverallQual: 7.5
Середні ціни в лівому та правому листках: [155840.27912867 299913.47027027]
```

Результати та інтерпретація

Для `OverallQual` можливі поділи виду «оцінка не більша за поріг» та «оцінка більша за поріг». Якщо присутні всі рівні від 1 до 10, достатньо перевірити дев'ять меж: 1.5, 2.5, ..., 9.5. Вивід показує фактичні рівні, кандидатів і обраний поділ. Обидві дочірні групи складаються із суміжних рівнів порядкової шкали.

`DecisionTreeRegressor` обробляє числову порядкову ознаку через порогові порівняння. Це використовує її порядок і не вимагає лінійного зв'язку оцінки з ціною. Для номінальних категорій довільні числові коди нав'язали б штучний порядок. Тому тут використовуємо `OneHotEncoder`: поділ даммі за 0.5 відокремлює один рівень від решти. Клас scikit-learn не перебирає довільні підмножини текстових категорій автоматично.

Окреме дерево обрало поріг 7.5: оцінки до 7 включно потрапляють ліворуч, від 8 праворуч. Середні ціни цих груп становлять приблизно 155840 та 299913 доларів.

2.г Обмеження розміру CART

Умова завдання

Очевидно, що без обмежень можна отримати дуже великі перенавчені дерева. Стрижка дерев (tree pruning) допомагає досягти розумного розміру. Підгоніть модель CART за допомогою DecisionTreeRegressor і обмежте її так, щоб вона мала не більше 10 розщеплень (параметр max_leaf_nodes=11).

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
tree = DecisionTreeRegressor(max_leaf_nodes=11, random_state=SEED)
tree.fit(Z_train, y_train)
structure = pd.Series({
    "Листків": tree.get_n_leaves(), "Поділів": tree.tree_.node_count - tree.get_n_leaves(),
    "Глибина": tree.get_depth(),
    "Навчальна MSE, дол.^2": mean_squared_error(y_train, tree.predict(Z_train)),
    "Валідаційна MSE, дол.^2": mean_squared_error(y_valid, tree.predict(Z_valid)),
})
shown_tree = deepcopy(tree)
scaler = prepared.named_transformers_["num"].named_steps["scale"]
for node, feature in enumerate(shown_tree.tree_.feature):
    if 0 <= feature < len(numeric_columns):
        shown_tree.tree_.threshold[node] = tree.tree_.threshold[node] * scaler.scale_[feature] +
            scaler.mean_[feature]
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

	Значення
Листків	11
Поділів	10
Глибина	4
Навчальна MSE, дол.^2	1.36773e+09
Валідаційна MSE, дол.^2	1.58884e+09

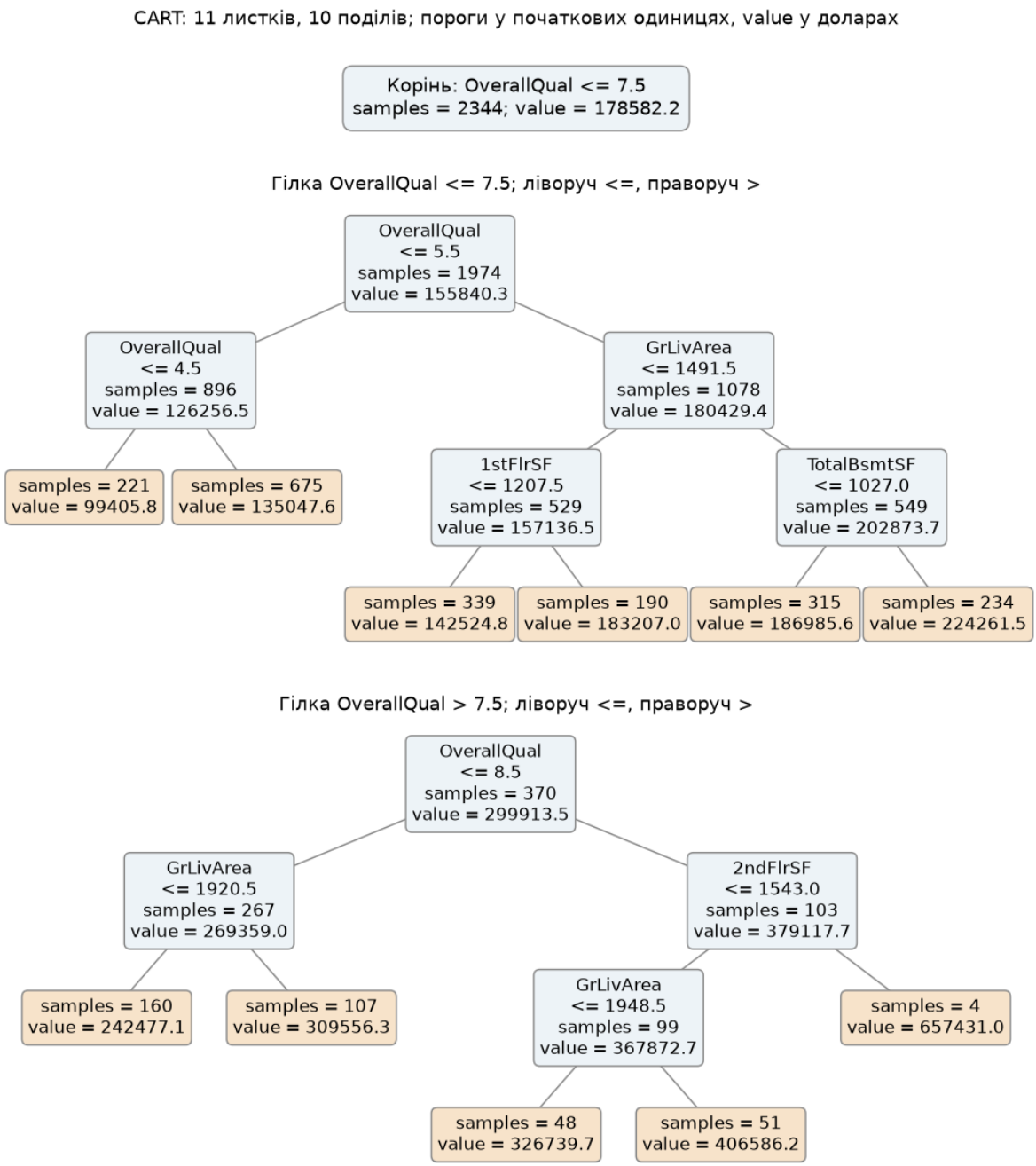


Рис. 3: Дерево CART із десятьма поділами. Пороги у початкових одиницях.

Результати та інтерпретація

`max_leaf_nodes=11` обмежує дерево одинадцятьма листками. Кожен бінарний поділ замінює один листок двома, тому кількість поділів дорівнює кількості листків мінус один. Числовий вивід і схема показують фактичний розмір дерева та його помилки.

Це обмеження росту під час навчання. На кожному кроці розщеплюється доступний листок із найбільшим покращенням критерію. Подальше обрізання через `ccp_alpha` розглядаємо в наступному пункті. На рисунку `samples` означає кількість навчальних будинків, а `value` середню ціну у вузлі.

Отримано 11 листків, 10 поділів і глибину 4. Навчальна MSE становить приблизно $1.368 \cdot 10^9$, валідаційна $1.589 \cdot 10^9$ дол.^{2}. Обмеження листків виконано; воно не гарантує, що саме цей розмір є найкращим.

2.д Обрізання дерева

Умова завдання

Знайдіть значення параметра складності (Cost-Complexity Pruning alpha, ccp_alpha) за допомогою методу cost_complexity_pruning_path() для дерева, навченого в попередньому пункті. Що це за параметр і що він регулює? Що станеться з архітектурою дерева, якщо ми суттєво збільшимо або зменшимо ccp_alpha? Перевірте це експериментально на ваших даних.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Прогноз: збільшення штрафу зменшить дерево; за дуже великого лишиться один листок.
print("Прогноз: зі зростанням ccp_alpha кількість листків не збільшується.")
pruning_path = tree.cost_complexity_pruning_path(Z_train, y_train)
pruning_rows = []
for alpha in pruning_path.ccp_alphas:
    model = clone(tree).set_params(ccp_alpha=float(alpha)).fit(Z_train, y_train)
    pruning_rows.append({
        "ccp_alpha": alpha, "Листків": model.get_n_leaves(), "Глибина": model.get_depth(),
        "Навчальна MSE, дол.^2": mean_squared_error(y_train, model.predict(Z_train)),
        "Валідаційна MSE, дол.^2": mean_squared_error(y_valid, model.predict(Z_valid)),
    })
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Прогноз: зі зростанням ccp_alpha кількість листків не збільшується.
```

ccp_alpha	Листків	Глибина	Навчальна MSE, дол.^2	Валідаційна MSE, дол.^2	Зважена неоднорідність шляху
0	11	4	1.36773e+09	1.58884e+09	1.36773e+09
6.7256e+07	10	4	1.43499e+09	1.73846e+09	1.43499e+09
7.9589e+07	9	4	1.51458e+09	1.83625e+09	1.51458e+09
8.59704e+07	8	3	1.60055e+09	1.93902e+09	1.60055e+09
9.023e+07	7	3	1.69078e+09	2.01528e+09	1.69078e+09
1.23087e+08	6	3	1.81386e+09	2.21125e+09	1.81386e+09
1.37522e+08	5	3	1.95139e+09	2.27441e+09	1.95139e+09
2.40431e+08	4	2	2.19182e+09	2.60159e+09	2.19182e+09
3.82003e+08	3	2	2.57382e+09	3.33575e+09	2.57382e+09
6.12612e+08	2	1	3.18643e+09	4.08078e+09	3.18643e+09
2.75931e+09	1	0	5.94574e+09	8.14008e+09	5.94574e+09

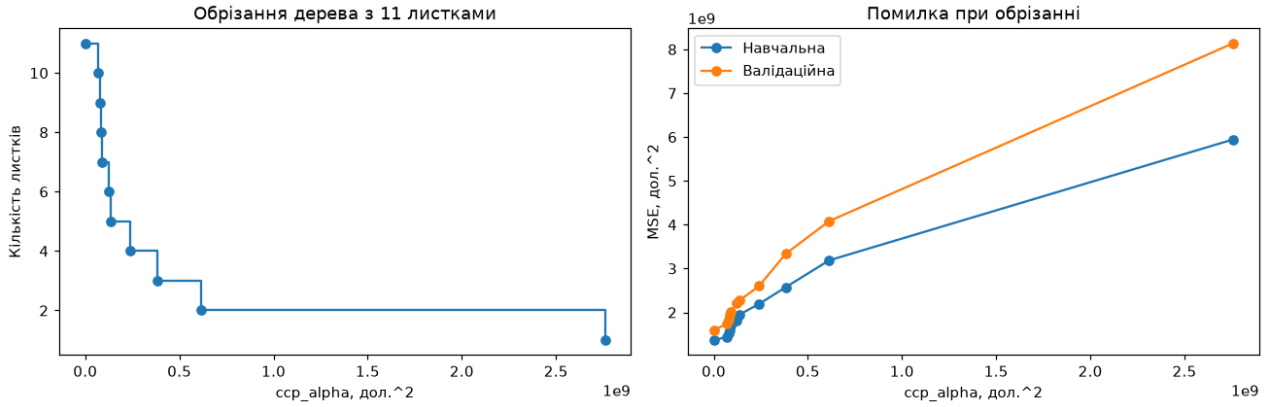


Рис. 4: Розмір дерева та помилка при обрізанні.

Результати та інтерпретація

Метод `cost_complexity_pruning_path()` дає послідовність критичних штрафів, за яких видаляються гілки. Це набір кандидатів, а не автоматичний вибір оптимального штрафу.

У scikit-learn мінімізується $R_\alpha(T) = R(T) + \alpha|T|$, де T є деревом, $|T|$ кількістю листків, α параметром `ccp_alpha`, а $R(T)$ сумою зважених неоднорідностей листків. Для критерію `squared_error` без ваг $R(T) = RSS(T)/n$, де $RSS(T)$ сума квадратів навчальних залишків, n кількість навчальних будинків. Тому `ccp_alpha` має шкалу дол.^{2}; у формулі зі звичайною сумою RSS штраф потрібно помножити на n.

За нульового штрафу залишається початкове дерево, обмежене одинадцятьма листками. Збільшення штрафу послідовно прибирає гілки; останній кандидат залишає лише корінь, що прогнозує навчальне середнє ціни. Зменшення штрафу повертає більш детальне дерево, але не скасовує обмеження `max_leaf_nodes=11`. Навчальна помилка при обрізанні не зменшується; поведінку валідаційної перевіряємо за таблицею.

Результати після обчислень

За нульового штрафу дерево мало 11 листків і глибину 4. Перший додатний поріг, приблизно $6.726 \cdot 10^7$, зменшив кількість листків до 10. За штрафу близько $2.759 \cdot 10^9$ залишився один листок, а глибина стала 0. На перевірених значеннях валідаційна MSE зростала, тому додаткове обрізання не покращило прогноз на цій валідаційній вибірці.

2.e Покращення в корені

Умова завдання

Перевірте значення покращення (improvement / зменшення критерію неоднорідності) у першому розщепленні дерева. Поясніть концепцію інформаційного прибутку (impurity reduction) та наведіть числовий вираз, за яким воно обчислюється в scikit-learn для першого вузла.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
t = tree.tree_
left, right = t.children_left[0], t.children_right[0]
N, NL, NR = t.weighted_n_node_samples[[0, left, right]]
I, IL, IR = t.impurity[[0, left, right]]
improvement = I - NL / N * IL - NR / N * IR
root_feature = feature_names[t.feature[0]]
root_threshold = shown_tree.tree_.threshold[0]
# Незалежна перевірка без використання збережених impurity дочірніх вузлів.
mask = Z_train[:, t.feature[0]] <= t.threshold[0]
root_rss = np.sum((y_train - y_train.mean()) ** 2)
children_rss = sum(np.sum((group - group.mean()) ** 2) for group in [y_train[mask],
    y_train[~mask]])
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Кореневий поділ: OverallQual <= 7.5
```

Вузол	Кількість	MSE, дол.^2
Корінь	2344	5.94574e+09
Лівий	1974	2.29321e+09
Правий	370	7.95189e+09

```
Покращення = 5945739180.175694 - (1974/2344) * 2293210777.136368 - (370/2344) * 7951889620.865341 = 2759306913.201561
дол.^2
Зменшення RSS, дол.^2: 6467815404544.459
```

Результати та інтерпретація

Інформаційний прибуток у корені показує, наскільки після поділу зменшується середній квадрат відхилень цін від середнього свого вузла. Від MSE кореня віднімаємо зважену MSE дочірніх вузлів.

$$\Delta = I - \frac{N_L}{N} I_L - \frac{N_R}{N} I_R.$$

Тут Δ є покращенням, I, I_L та I_R є MSE кореня, лівого й правого вузлів, а N, N_L та N_R позначають кількість навчальних будинків у цих вузлах.

У корені обрано поділ `OverallQual ≤ 7.5` у початковій шкалі. Підставляємо значення зі збереженого виводу.

$$\Delta \approx \left(5.94574 - \frac{1974}{2344} \cdot 2.29321 - \frac{370}{2344} \cdot 7.95189\right) \cdot 10^9 \approx 2.7593 \cdot 10^9 \text{ дол.}^2.$$

MSE правого вузла вища за MSE кореня. Проте в ньому лише 370 із 2344 будинків, і зважена MSE двох дочірніх вузлів після поділу зменшилася.

2.ж Важливості дерева та коефіцієнти

Умова завдання

Обчисліть важливість ознак (feature_importances_) для дерева регресії та порівняйте топ-10 результатів із коефіцієнтами, отриманими в регресії ласо та класичній лінійній регресії.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Відтворюємо фінальні Lasso та OLS частини (1), не читаючи її стан ядра.
lasso_search = GridSearchCV(
    Pipeline([("prepare", clone(preprocessor)), ("model", Lasso(max_iter=50000, tol=1e-6))]),
    {"model__alpha": np.logspace(1, 4, 35)},
    cv=KFold(n_splits=5, shuffle=True, random_state=SEED),
    scoring="neg_mean_squared_error", n_jobs=1,
).fit(X_train, y_train)
lasso_model = lasso_search.best_estimator_.named_steps["model"]
ols_model = LinearRegression().fit(Z_train, y_train)
importance = pd.DataFrame({"Lasso, коеф.": lasso_model.coef_, "OLS, коеф.": ols_model.coef_,
                           "CART MDI": tree.feature_importances_},
                          index=pd.Index(feature_names, name="Ознака"))
tree_top10 = importance.sort_values("CART MDI", ascending=False, kind="stable").head(10)
# Перевіряємо MDI через зважені зменшення неоднорідності всіх поділів.
manual_mdi = np.zeros(Z_train.shape[1])
for node, feature in enumerate(t.feature):
    if feature >= 0:
        l, r = t.children_left[node], t.children_right[node]
        manual_mdi[feature] += (t.weighted_n_node_samples[node] * t.impurity[node]
                                - t.weighted_n_node_samples[l] * t.impurity[l]
                                - t.weighted_n_node_samples[r] * t.impurity[r])
manual_mdi /= manual_mdi.sum()
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Обраний alpha Lasso: 76.26985859023443
Ненульових коефіцієнтів Lasso: 126
Ранг OLS / ознак: 280 / 294
```

Ознака	Lasso, коеф.	OLS, коеф.	CART MDI
OverallQual	14157.1	8711.49	0.8397
GrLivArea	23057.6	16429.2	0.0940963
2ndFlrSF	4383.1	13632.9	0.0300397
1stFlrSF	0	6470.09	0.018779
TotalBsmtSF	4197.38	9661.19	0.0173851
LotFrontage	-1624.16	1350.95	0
LotArea	3303.43	5462.63	0
OverallCond	6306.27	6698.35	0
YearBuilt	9510.14	12549.5	0
YearRemodAdd	2077.11	1021.81	0

```
Кількість ознак із додатною CART MDI: 5
```

CART MDI	Lasso за модулем	OLS за модулем
OverallQual	PoolQC_Gd	RoofMatl_Membran
GrLivArea	MiscFeature_Gar2	RoofMatl_WdShngl
2ndFlrSF	RoofMatl_WdShngl	RoofMatl_Metal
1stFlrSF	Neighborhood_StoneBr	RoofMatl_Roll
TotalBsmtSF	Neighborhood_NoRidge	RoofMatl_WdShake
LotFrontage	Neighborhood_NridgHt	RoofMatl_CompShg
LotArea	KitchenQual_TA	RoofMatl_Tar&Grv
OverallCond	Neighborhood_GrnHill	MiscFeature_Othr
YearBuilt	KitchenQual_Gd	MiscFeature_Gar2
YearRemodAdd	GrLivArea	MiscFeature_Shed

Результати та інтерпретація

MDI підсумовує зважене зменшення неоднорідності в усіх поділах за ознакою та нормує внески так, щоб їхня сума дорівнювала одиниці. У таблиці зіставляємо MDI десяти ознак дерева з коефіцієнтами тих самих ознак у Lasso й МНК. MDI безрозмірна й показує внесок у поділи. Знак коефіцієнта показує напрям зміни прогнозу лінійної моделі за незмінних інших ознак.

Лише п’ять ознак мають додатну MDI. Решта рядків топ-10 мають нульову MDI нарівні з іншими невикористаними ознаками. Найвищі MDI у `OverallQual` (0.8397) та `GrLivArea` (0.09410), і для обох коефіцієнти Lasso й МНК додатні. Дерево використало `1stFlrSF`, а Lasso оцінило її коефіцієнт нулем.

2.i Помилка ансамблів

Умова завдання

Використовуйте беггінг (BaggingRegressor) та випадковий ліс (RandomForestRegressor) для прогнозування SalePrice. Побудуйте графік залежності середньоквадратичної помилки (MSE) на навчальній та валідаційній вибірках від кількості базових дерев (n_estimators). Прокоментуйте поведінку обох кривих на графіку.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Прогноз до досліду: усереднення зменшить коливання прогнозів; криві можуть мати нерівності.
print("Прогноз: зі збільшенням ансамблю валідаційна MSE стабілізуватиметься; монотонність не гарантована.")
ensemble_sizes = [1, 5, 10, 25, 50, 100, FINAL_TREES]
ensembles = {
    "Bagging": BaggingRegressor(
        estimator=DecisionTreeRegressor(random_state=SEED),
        bootstrap=True, max_samples=1.0, max_features=1.0,
        warm_start=True, random_state=SEED, n_jobs=1),
    "Random Forest": RandomForestRegressor(
        bootstrap=True, max_features=1/3, warm_start=True,
        random_state=SEED, n_jobs=1),
}
curve_rows = []
for name, model in ensembles.items():
    for size in ensemble_sizes:
        # Нові дерева додаються до того самого ансамблю; старі залишаються.
        model.set_params(n_estimators=size).fit(Z_train, y_train)
        curve_rows.append({"Модель": name, "Дерев": size,
                           "Навчальна MSE, дол.^2": mean_squared_error(y_train, model.predict(Z_train)),
                           "Валідаційна MSE, дол.^2": mean_squared_error(y_valid, model.predict(Z_valid))})
curves = pd.DataFrame(curve_rows)
# Описовий критерій плато: не більш ніж на 1% вище MSE при 200 деревах.
plateau_rows = []
for name, group in curves.groupby("Модель", sort=False):
    mse = group["Валідаційна MSE, дол.^2"].to_numpy()
    acceptable = np.array([np.all(mse[i:] <= 1.01 * mse[-1]) for i in range(len(mse))])
    plateau_rows.append({"Модель": name, "Початок плато на сітці": group["Дерев"].to_numpy()[acceptable][0],
                         "MSE при 200 деревах, дол.^2": mse[-1]})
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Прогноз: зі збільшенням ансамблю валідаційна MSE стабілізуватиметься; монотонність не гарантована.
```

Модель	Дерев	Навчальна MSE, дол.^2	Валідаційна MSE, дол.^2
Bagging	1	6.53192e+08	1.53168e+09
Bagging	5	2.05021e+08	8.4875e+08
Bagging	10	1.68949e+08	7.66191e+08
Bagging	25	1.22267e+08	7.41785e+08
Bagging	50	1.07443e+08	7.22265e+08
Bagging	100	1.00961e+08	7.05429e+08
Bagging	200	9.85243e+07	7.0914e+08
Random Forest	1	6.40523e+08	2.56631e+09
Random Forest	5	2.00007e+08	9.50028e+08
Random Forest	10	1.50852e+08	8.11277e+08
Random Forest	25	1.21991e+08	7.24634e+08
Random Forest	50	1.05962e+08	7.25563e+08
Random Forest	100	9.81004e+07	6.97905e+08
Random Forest	200	9.43458e+07	7.18452e+08

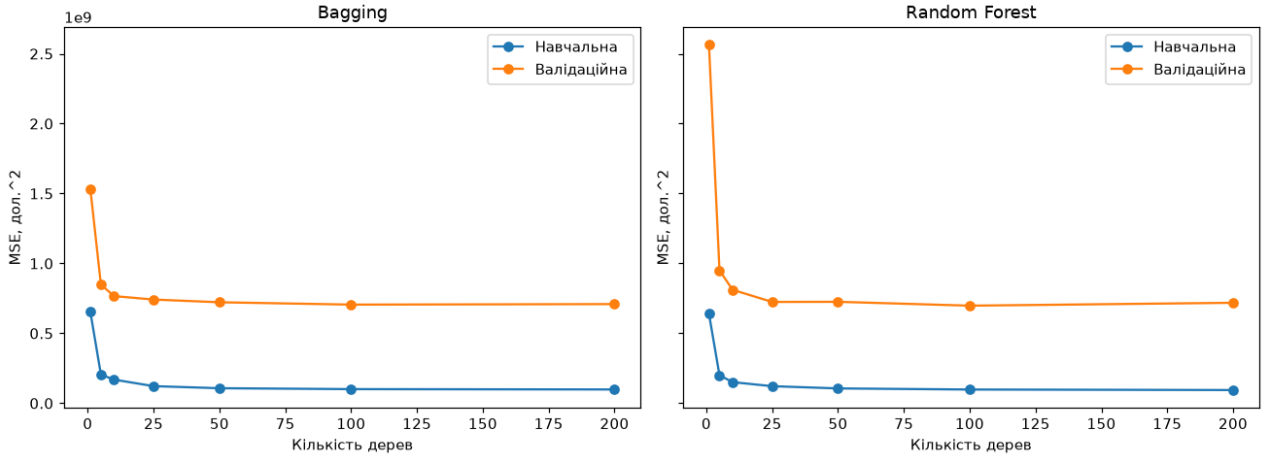


Рис. 5: Навчальна й валідаційна MSE залежно від кількості дерев.

Модель	Початок плато на сітці	MSE при 200 деревах, дол.^2
Bagging	100	7.0914e+08
Random Forest	25	7.18452e+08

Результати та інтерпретація

Обидва ансамблі усереднюють прогнози дерев, навчених на вибірках із поверненням. У випадковому лісі кожен вузол розглядає випадкову підмножину ознак, тому дерева можуть давати менш схожі прогнози.

Початком плато називаємо найменшу перевірену кількість дерев, від якої всі показані валідаційні MSE не більш ніж на 1% перевищують MSE цієї моделі при 200 деревах.

Навчальна MSE спадала на всій перевіреній сітці в обох моделей і залишалася нижчою за валідаційну. Валідаційна MSE найбільше впала від 1 до 5 дерев. В обох моделей вона була найменшою при 100 деревах і трохи зросла при 200. За критерієм 1% плато починається зі 100 дерев для беггінгу і з 25 для випадкового лісу. При 200 деревах валідаційна MSE становить $7.0914 \cdot 10^8$ дол.² для беггінгу та $7.1845 \cdot 10^8$ дол.² для лісу.

2.к Час та важливості моделей

Умова завдання

Порівняйте час виконання (time.time()) обох ансамблевих підходів. Чому випадковий ліс зазвичай обчислювально швидший і потребує менше дерев порівняно з чистим беггінгом над глибокими деревами? Перевірте вбудований метод розрахунку важливості ознак для RandomForestRegressor (MDI або permutation importance). Зробіть фінальну порівняльну таблицю важливостей змінних для всіх моделей: Lasso, OLS, одиночне дерево та Random Forest.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# Прогноз: ліс із третиною ознак має навчатися швидше; перевіряємо фактичний час.
print("Прогноз: Random Forest із max_features=1/3 швидший за беггінг з усіма ознаками.")
timing_rows = []
for repeat in range(3):
    # Чергуємо порядок, щоб один метод не завжди виконувався першим.
    names = list(ensembles) if repeat % 2 == 0 else list(reversed(ensembles))
    for name in names:
        model = clone(ensembles[name]).set_params(warm_start=False, n_estimators=FINAL_TREES)
        start = time.time()
        model.fit(Z_train, y_train)
        elapsed = time.time() - start
        timing_rows.append({"Повтор": repeat + 1, "Модель": name, "Час навчання, с": elapsed})
        # Незалежне навчання з нуля відтворює ансамбль, нарощений у (i).
        assert np.allclose(model.predict(Z_valid), ensembles[name].predict(Z_valid))
timings = pd.DataFrame(timing_rows)
timing_summary = timings.groupby("Модель")["Час навчання, с"].agg(["median", "min", "max"])
forest = ensembles["Random Forest"]
importance["Random Forest MDI"] = forest.feature_importances_
# Відтворюємо MDI лісу за нормованими важливостями окремих дерев.
forest_mdi = np.mean([
    estimator.feature_importances_
    for estimator in forest.estimators_
    if estimator.tree_.node_count > 1
], axis=0)
forest_mdi /= forest_mdi.sum()
mdi_difference = np.abs(forest_mdi - forest.feature_importances_)
# Повна таблиця зберігає знак коефіцієнтів і не змішує їх одиниці з MDI.
final_importance = importance.sort_values("Random Forest MDI", ascending=False)
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Прогноз: Random Forest із max_features=1/3 швидший за беггінг з усіма ознаками.
```

Повтор	Модель	Час навчання, с
1	Bagging	5.72391
1	Random Forest	2.11956
2	Random Forest	2.10959
2	Bagging	5.63344
3	Bagging	5.63201
3	Random Forest	2.12593

Модель	median	min	max
Bagging	5.63344	5.63201	5.72391
Random Forest	2.11956	2.10959	2.12593

```
Відношення медіанного часу Bagging / Random Forest: 2.6578315085294992
Максимальна абсолютна різниця з MDI лісу: 0.0
```

Ознака	Lasso, коеф.	OLS, коеф.	CART MDI	Random Forest MDI
OverallQual	14157.1	8711.49	0.8397	0.286514
GrLivArea	23057.6	16429.2	0.0940963	0.104882
GarageCars	6388.78	3417.26	0	0.101298
YearBuilt	9510.14	12549.5	0	0.0718618
TotalBsmtSF	4197.38	9661.19	0.0173851	0.0483594
ExterQual_TA	-10256	-18206.5	0	0.0451815
GarageArea	589.571	2437.46	0	0.04317
1stFlrSF	0	6470.09	0.018779	0.0391335
2ndFlrSF	4383.1	13632.9	0.0300397	0.0225395
FullBath	2620.66	1406.64	0	0.020463
BsmtFinSF1	4262.43	9812.54	0	0.0197775
LotArea	3303.43	5462.63	0	0.0188103
YearRemodAdd	2077.11	1021.81	0	0.0122537
Fireplaces	3818.73	3757.38	0	0.0113411
KitchenQual_TA	-24421.7	-24433.6	0	0.0099555
FireplaceQu_NA	-0	-1791.63	0	0.00784692
MasVnrArea	2600.72	3286.45	0	0.00765501
Foundation_PConc	2483.76	1431.63	0	0.00656926
LotFrontage	-1624.16	1350.95	0	0.00587695
BsmtUnfSF	-0	744.428	0	0.00550854

Скорочено: із 294 ознак показано 20 з найбільшою MDI випадкового лісу; решта 274 мають меншу MDI.

```
Суми MDI: {'CART MDI': 0.9999999999999998, 'Random Forest MDI': 0.9999999999999999}
```

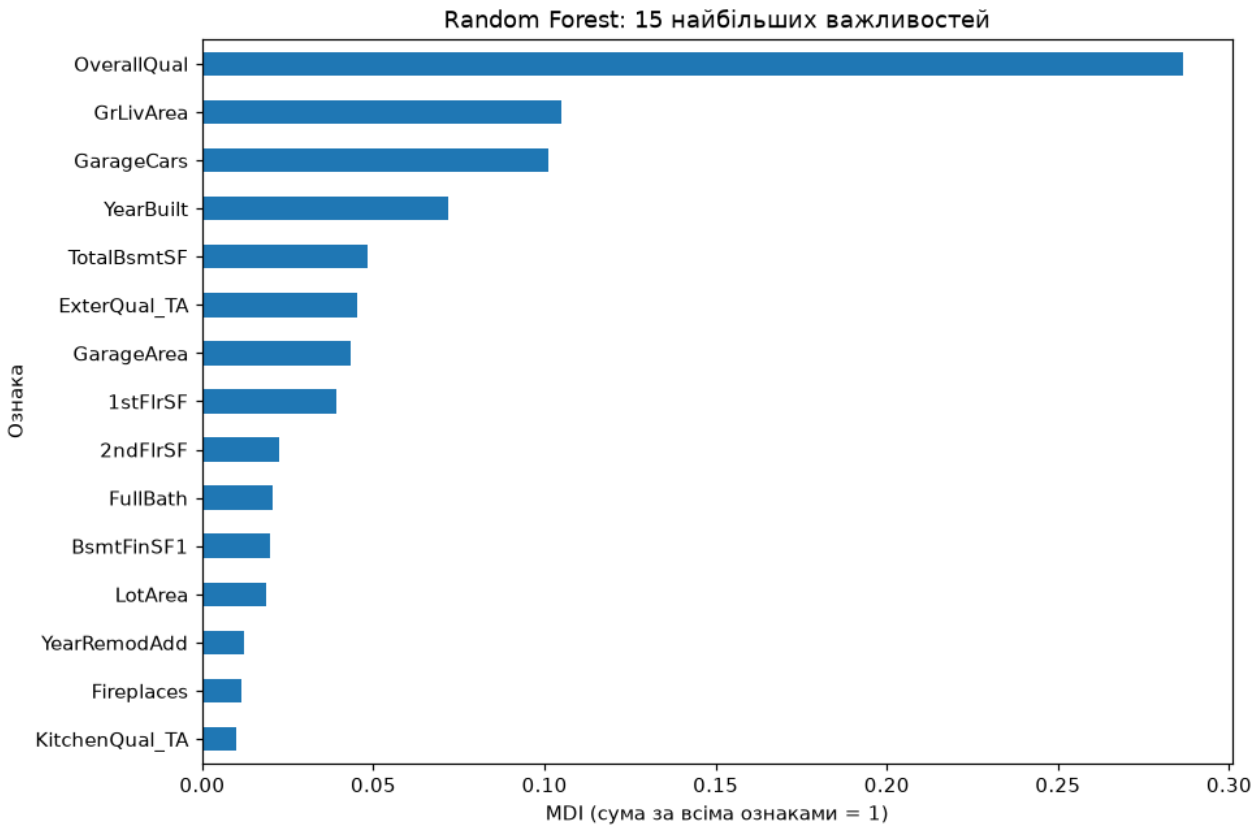


Рис. 6: П’ятнадцять найбільших важливостей випадкового лісу.

У цьому запуску медіанний час навчання становить 5.633 с для беггінгу та 2.120 с для випадкового лісу. Ліс швидший приблизно у 2.66 раза. Це підтверджує початкове очікування для заданих налаштувань.

Результати та інтерпретація

Через `time.time()` вимірюємо час навчання з нуля. Для обох методів використовуємо 200 глибоких дерев, ті самі дані й один потік. Порівнюємо медіани трьох повторів із чергуванням порядку.

Беггінг розглядає всі ознаки у вузлі, а ліс із `max_features=1/3` зазвичай перевіряє менше кандидатів, тому кожен поділ можна знайти швидше. Випадковий вибір ознак також зменшує подібність дерев. Усереднення менш схожих прогнозів може стабілізувати помилку за меншої кількості дерев.

Щоб перевірити MDI лісу, усереднюємо важливості дерев із поділами, нормуємо їхню суму до одиниці й порівнюємо значення для кожної ознаки з `feature_importances_`. Фінальна таблиця зіставляє коефіцієнти Lasso й МНК із MDI одиночного дерева та лісу.

За критерієм 1% із 2.і ліс досяг плато раніше за беггінг, хоча при 200 деревах його валідаційна MSE трохи вища. Найвищі MDI лісу мають `OverallQual` (0.2865), `GrLivArea` (0.1049) і `GarageCars` (0.1013). Частка `OverallQual` у лісі менша, ніж в одиночному дереві, де її MDI становить 0.8397.

3. Логістична регресія

Перейдемо до задачі бінарної класифікації. Визначимо цільову змінну:

$$Y_i = \begin{cases} 1, & \text{якщо SalePrice} > 130000 \\ 0, & \text{в іншому випадку.} \end{cases}$$

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
y = (df["SalePrice"] > 130000).astype(int).rename("Above130000")
excluded = ["SalePrice", "Order", "PID", "MSSubClass", "GarageYrBlt", "GrLivArea", "TotalBsmtSF"]
X = df.select_dtypes(include="number").drop(columns=excluded)
X_train, X_valid, y_train, y_valid = train_test_split(X, y, test_size=0.2, random_state=SEED)
imputer = SimpleImputer(strategy="median")
raw_train = pd.DataFrame(imputer.fit_transform(X_train), index=X_train.index, columns=X.columns)
raw_valid = pd.DataFrame(imputer.transform(X_valid), index=X_valid.index, columns=X.columns)
scaler = StandardScaler()
Z_train = pd.DataFrame(scaler.fit_transform(raw_train), index=X_train.index, columns=X.columns)
Z_valid = pd.DataFrame(scaler.transform(raw_valid), index=X_valid.index, columns=X.columns)
Z_train = sm.add_constant(Z_train)
Z_valid = sm.add_constant(Z_valid)
means = pd.Series(scaler.mean_, index=X.columns)
scales = pd.Series(scaler.scale_, index=X.columns)
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Клас	Навчальна	Валідаційна
0	627	148
1	1717	438

```
Початкових числових ознак: 32
Ранг / стовпців із константою: 33 33
Початкові ознаки: ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'MasVnrArea',
'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'BsmtFullBath', 'BsmtHalfBath',
'FullBath', 'HalfBath', 'BedroomAbvGr', 'KitchenAbvGr', 'TotRmsAbvGrd', 'Fireplaces', 'GarageCars', 'GarageArea',
'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch', 'PoolArea', 'MiscVal', 'MoSold', 'YrSold']
```

	Заповнено медіаною у навчальній частині
LotFrontage	393
MasVnrArea	19
BsmtFinSF1	1
BsmtFinSF2	1
BsmtUnfSF	1
BsmtFullBath	1
BsmtHalfBath	1
GarageCars	1
GarageArea	1

3.a Відбір за AIC

Умова завдання

Оцініть логістичну регресію, використовуючи покроковий зворотний відбір (Backward Selection) за інформаційним критерієм AIC. Для реалізації цього алгоритму та зручного розрахунку AIC використовуйте модуль statsmodels.formula.api.logit або sm.Logit. Щоб прискорити обчислення, почніть покроковий відбір лише з числових ознак (features). Надалі аналізуйте виключно фінальну підібрану модель.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
def fit_logit(columns):
    # Усі порівняння використовують ті самі рядки й звичайний ММП без штрафу.
    result = sm.Logit(y_train, Z_train[["const"] + columns]).fit(
        method="newton", maxiter=200, tol=1e-9, disp=False)
    if not result.mle_retvals["converged"] or not np.isfinite(result.aic):
        raise RuntimeError("Кандидатна модель не збіглася; AIC не порівнюємо.")
    return result
selected = list(X.columns)
model = fit_logit(selected)
selection_rows = [{"Крок": 0, "Вилучено": "Початкова модель", "Ознак": len(selected), "AIC":
     model.aic}]
fits_checked = 1
while selected:
    candidates = []
    for column in selected:
        remaining = [name for name in selected if name != column]
        candidate = fit_logit(remaining)
        fits_checked += 1
        candidates.append((candidate.aic, column, candidate))
    best_aic, removed, candidate = min(candidates, key=lambda item: item[0])
    # Поріг відокремлює покращення від похибки округлення.
    if best_aic >= model.aic - 1e-8:
        break
    selected.remove(removed)
    model = candidate
    selection_rows.append({"Крок": len(selection_rows), "Вилучено": removed,
                           "Ознак": len(selected), "AIC": model.aic})
# Повертаємо нахили та константу до початкових одиниць.
beta_raw = model.params[selected] / scales[selected]
intercept_raw = model.params["const"] - (beta_raw * means[selected]).sum()
se_raw = model.bse[selected] / scales[selected]
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Крок	Вилучено	Ознак	AIC
0	Початкова модель	32	975.48311
1	MiscVal	31	973.60217
2	OpenPorchSF	30	971.72607
3	TotRmsAbvGrd	29	969.91267
4	BedroomAbvGr	28	968.06524
5	MoSold	27	966.41001
6	MasVnrArea	26	964.86476
7	3SsnPorch	25	963.71709
8	LowQualFinSF	24	963.12802

```
Оцінених моделей зі збіжністю: 253
Відібрані ознаки: ['LotFrontage', 'LotArea', 'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd', 'BsmtFinSF1',
'BsmtFinSF2', 'BsmtUnfSF', '1stFlrSF', '2ndFlrSF', 'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
'KitchenAbvGr', 'Fireplaces', 'GarageCars', 'GarageArea', 'WoodDeckSF', 'EnclosedPorch', 'ScreenPorch', 'PoolArea',
'YrSold']
Найкращий AIC після ще одного вилучення: 963.4442696643245
Фінальний AIC: 963.128015126287
Summary: коефіцієнти для стандартизованих ознак.
                           Logit Regression Results
==============================================================================
Dep. Variable:            Above130000   No. Observations:                 2344
Model:                          Logit   Df Residuals:                     2319
Method:                           MLE   Df Model:                           24
Date:                Mon, 09 Nov 2026   Pseudo R-squ.:                  0.6646
Time:                        12:53:07   Log-Likelihood:                -456.56
converged:                       True   LL-Null:                       -1361.3
Covariance Type:            nonrobust   LLR p-value:                     0.000
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.1408      0.229     18.093      0.000       3.692       4.589
LotFrontage    0.4373      0.142      3.089      0.002       0.160       0.715
LotArea        0.6398      0.260      2.461      0.014       0.130       1.149
OverallQual    1.1347      0.174      6.520      0.000       0.794       1.476
OverallCond    0.8269      0.109      7.565      0.000       0.613       1.041
YearBuilt      1.4924      0.167      8.922      0.000       1.165       1.820
YearRemodAdd   0.3679      0.103      3.576      0.000       0.166       0.569
BsmtFinSF1     1.1939      0.216      5.540      0.000       0.772       1.616
BsmtFinSF2     0.3983      0.112      3.571      0.000       0.180       0.617
BsmtUnfSF      0.8548      0.170      5.041      0.000       0.522       1.187
1stFlrSF       1.5843      0.225      7.029      0.000       1.143       2.026
2ndFlrSF       1.8117      0.219      8.264      0.000       1.382       2.241
BsmtFullBath   0.4714      0.118      3.990      0.000       0.240       0.703
BsmtHalfBath   0.1232      0.082      1.505      0.132      -0.037       0.284
FullBath       0.3786      0.148      2.558      0.011       0.088       0.669
HalfBath       0.2386      0.118      2.023      0.043       0.007       0.470
KitchenAbvGr  -0.4876      0.088     -5.527      0.000      -0.660      -0.315
Fireplaces     0.3998      0.109      3.682      0.000       0.187       0.613
GarageCars     0.3209      0.200      1.607      0.108      -0.071       0.712
GarageArea     0.3148      0.205      1.538      0.124      -0.086       0.716
WoodDeckSF     0.2712      0.104      2.609      0.009       0.067       0.475
EnclosedPorch  0.1693      0.091      1.855      0.064      -0.010       0.348
ScreenPorch    0.4744      0.131      3.633      0.000       0.218       0.730
PoolArea      -0.2107      0.095     -2.220      0.026      -0.397      -0.025
YrSold        -0.1396      0.085     -1.646      0.100      -0.306       0.027
==============================================================================

Possibly complete quasi-separation: A fraction 0.20 of observations can be
perfectly predicted. This might indicate that there is complete
quasi-separation. In this case some parameters will not be identified.
```

	Значення
Частка |y-p| < 0.0001	0.19837884
Максимальний модуль градієнта	5.3290705e-14
Найменше власне значення інформаційної матриці	8.7662306
Максимальна різниця коефіцієнтів Newton і BFGS	1.2407918e-07
Різниця log-likelihood Newton і BFGS	5.0590643e-12

	Коефіцієнт у початкових одиницях	Стандартна похибка	p-значення
LotFrontage	0.020470732	0.0066260011	0.002005241
LotArea	7.9491266e-05	3.2299513e-05	0.013852448
OverallQual	0.81739431	0.12537262	7.0438966e-11
OverallCond	0.74800588	0.098873607	3.8704935e-14
YearBuilt	0.04919673	0.0055143401	4.5960498e-19

	Коефіцієнт у початкових одиницях	Стандартна похибка	p-значення
YearRemodAdd	0.017700964	0.0049495669	0.00034853813
BsmtFinSF1	0.002641284	0.0004767892	3.0293639e-08
BsmtFinSF2	0.0023383025	0.00065487033	0.00035611781
BsmtUnfSF	0.0019714091	0.00039104058	4.6203265e-07
1stFlrSF	0.004114672	0.00058539937	2.0827438e-12
2ndFlrSF	0.0042424299	0.00051338588	1.4131057e-16
BsmtFullBath	0.89737154	0.22490109	6.6053266e-05
BsmtHalfBath	0.5101582	0.33907794	0.13244096
FullBath	0.6857339	0.26809045	0.010532292
HalfBath	0.47773137	0.23618601	0.043104991
KitchenAbvGr	-2.3075442	0.41753967	3.2663239e-08
Fireplaces	0.61245528	0.1663517	0.00023169371
GarageCars	0.43007773	0.2677032	0.10815381
GarageArea	0.0014827435	0.00096385208	0.12396265
WoodDeckSF	0.0021723264	0.00083278367	0.009093675
EnclosedPorch	0.0025260833	0.0013616231	0.063567669
ScreenPorch	0.0085011718	0.0023399359	0.0002800593
PoolArea	-0.005498648	0.0024770838	0.026431873
YrSold	-0.10616134	0.06450283	0.09979671

```
Константа у початкових одиницях: 63.877265241644466
```

Результати та інтерпретація

Для логістичної регресії оцінюємо коефіцієнти методом максимальної правдоподібності через `sm.Logit`. Критерій $AIC = -2\ell + 2k$ поєднує відповідність даним і штраф за складність. Тут ℓ є логарифмом правдоподібності оціненої моделі, а k кількістю параметрів разом із вільним членом.

Починаємо з усіх підготовлених числових ознак. На кожному кроці по черзі вилучаємо по одній ознаці й повторно оцінюємо модель. Приймаємо вилучення з найменшим AIC, якщо він нижчий за поточний. Коли жодне таке вилучення не знижує AIC, зупиняємося. Вільний член зберігаємо на всіх кроках.

`summary()` показує коефіцієнти стандартизованих ознак. Для подальшої інтерпретації повертаємо їх у початкові одиниці за формулами $b_j = \widehat{\beta}_j / s_j$ та $b_0 = \widehat{\beta}_0 - \sum_j b_j \bar{x}_j$. Тут $\widehat{\beta}_j$ і $\widehat{\beta}_0$ є коефіцієнтом ознаки j та вільним членом після стандартизації, s_j і $\bar{x}_j$ її навчальним масштабом і середнім, а b_j і b_0 відповідними коефіцієнтами в початкових одиницях.

Відбір залишив 24 із 32 ознак. AIC зменшився з 975.48311 до 963.12802. Найкраще наступне вилучення дало б AIC 963.44427, тому відбір зупинився. Оптимізація досягла збіжності в усіх 253 оцінюваннях, включно з початковою моделлю.

`summary()` попереджає про можливе квазірозділення через велику частку прогнозів, близьких до фактичних 0 або 1. Newton і BFGS дали практично однакові оцінки: найбільша абсолютна різниця коефіцієнтів становила близько $1.24 \cdot 10^{-7}$.

3.б Відношення шансів

Умова завдання

Розгляньте коефіцієнт біля змінної LotArea. Його не можна інтерпретувати лінійно. Наведіть правильну економічну інтерпретацію через відношення шансів (Odds Ratio), експоненціювавши коефіцієнт: $\exp(\beta_{\text{LotArea}})$.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
lot_beta = beta_raw["LotArea"]
lot_ci = model.conf_int().loc["LotArea"] / scales["LotArea"]
lot_rows = []
for increase in [1, 1000]:
    odds_ratio = np.exp(increase * lot_beta)
    lot_rows.append({"Приріст площі, кв. футів": increase, "beta на 1 кв. фут": lot_beta,
                     "Відношення шансів": odds_ratio, "Зміна шансів, %": 100 * (odds_ratio - 1),
                     "Нижня межа OR, 95%": np.exp(increase * lot_ci.iloc[0]),
                     "Верхня межа OR, 95%": np.exp(increase * lot_ci.iloc[1])})
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Приріст площі, кв. футів	beta на 1 кв. фут	Відношення шансів	Зміна шансів, %	Нижня межа OR, 95%	Верхня межа OR, 95%
1	7.9491266e-05	1.0000795	0.0079494425	1.0000162	1.0001428
1000	7.9491266e-05	1.0827361	8.2736103	1.0163171	1.1534958

Результати та інтерпретація

Шанси ціни понад 130000 доларів дорівнюють $p/(1-p)$, де p є ймовірністю перевищення цього порогу. За незмінних інших ознак приріст площі ділянки множить шанси на $OR = e^{db}$, де OR є відношенням шансів, d приростом площі у квадратних футах, а b коефіцієнтом `LotArea` на один квадратний фут. Тут e є основою натурального логарифма.

Додатковий квадратний фут множить шанси на 1.0000795. Додаткові 1000 квадратних футів множать їх на 1.0827361, тобто збільшують шанси приблизно на 8.27% за незмінних інших ознак.

3.в Значущість і знаки

Умова завдання

Сформуйте списки змінних, які статистично значуще збільшують ймовірність події $Y = 1$ (додатній коефіцієнт), та тих, що її зменшують (від'ємний коефіцієнт). Чи узгоджуються ці знаки з базовою економічною логікою ринку нерухомості?

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
significant_positive = coefficient_table[(beta_raw > 0) & (model.pvalues[selected] <
    SIGNIFICANCE)]
significant_negative = coefficient_table[(beta_raw < 0) & (model.pvalues[selected] <
    SIGNIFICANCE)]
# Перевіряємо, скільки навчальних прикладів підтримують висновок щодо басейну.
display(pd.crosstab(X_train["PoolArea"] > 0, y_train).rename_axis("Площа басейну > 0"))
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

```
Статистично значущі додатні коефіцієнти:
```

	Коефіцієнт у початкових одиницях	Стандартна похибка	p-значення
LotFrontage	0.020470732	0.0066260011	0.002005241
LotArea	7.9491266e-05	3.2299513e-05	0.013852448
OverallQual	0.81739431	0.12537262	7.0438966e-11
OverallCond	0.74800588	0.098873607	3.8704935e-14
YearBuilt	0.04919673	0.0055143401	4.5960498e-19
YearRemodAdd	0.017700964	0.0049495669	0.00034853813
BsmtFinSF1	0.002641284	0.0004767892	3.0293639e-08
BsmtFinSF2	0.0023383025	0.00065487033	0.00035611781
BsmtUnfSF	0.0019714091	0.00039104058	4.6203265e-07
1stFlrSF	0.004114672	0.00058539937	2.0827438e-12
2ndFlrSF	0.0042424299	0.00051338588	1.4131057e-16
BsmtFullBath	0.89737154	0.22490109	6.6053266e-05
FullBath	0.6857339	0.26809045	0.010532292
HalfBath	0.47773137	0.23618601	0.043104991
Fireplaces	0.61245528	0.1663517	0.00023169371
WoodDeckSF	0.0021723264	0.00083278367	0.009093675
ScreenPorch	0.0085011718	0.0023399359	0.0002800593

```
Статистично значущі від'ємні коефіцієнти:
```

	Коефіцієнт у початкових одиницях	Стандартна похибка	p-значення
KitchenAbvGr	-2.3075442	0.41753967	3.2663239e-08
PoolArea	-0.005498648	0.0024770838	0.026431873

```
Відібрані, але незначущі на рівні 0.05: ['BsmtHalfBath', 'GarageCars', 'GarageArea', 'EnclosedPorch', 'YrSold']
```

Above130000	0	1
False	626	1706
True	1	11

Результати та інтерпретація

Логістична функція зростає, тому за незмінних інших ознак знак коефіцієнта визначає напрям зміни прогнозованої ймовірності класу 1. У таблицях наведено ознаки з додатними й від'ємними коефіцієнтами та p-значеннями меншими за 0.05. Ці p-значення умовні для відібраної моделі й не враховують відбір за AIC.

Додатні коефіцієнти узгоджуються з ринковою логікою: за інших рівних умов якісніші, краще збережені, більші або новіші будинки частіше перевищують ціновий поріг. Значущі від'ємні коефіцієнти мають `KitchenAbvGr` і `PoolArea`. Це умовні зв'язки в моделі. Результат для `PoolArea` варто тлумачити обережно, бо лише 12 навчальних будинків мають додатну площу басейну.

3.г П’ять прогнозів

Умова завдання

Візьміть випадкові 5 будинків із вибірки та розрахуйте для них ймовірність належати до класу 1 за допомогою методу .predict(). Запишіть повну математичну формулу логістичної функції $p = \frac{1}{1+e^{-z}}$ для першого будинку, підставивши туди реальні значення його ознак та оцінені моделлю коефіцієнти. Який найпростіший спосіб перейти від ймовірностей до безпосереднього прогнозу класу (класичний поріг cutoff)?

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
# П'ять будинків із валідаційної частини, що не брали участі у відборі моделі.
sample_index = raw_valid.sample(n=5, random_state=SEED).index
sample_design = Z_valid.loc[sample_index, ["const"] + selected]
sample_probabilities = model.predict(sample_design)
sample_results = df.loc[sample_index, ["Order", "SalePrice"]].copy()
sample_results["Фактичний клас"] = y_valid.loc[sample_index]
sample_results["P(Y=1)"] = sample_probabilities
sample_results["Клас за порогом 0.5"] = (sample_probabilities >= 0.5).astype(int)
first_index = sample_index[0]
first_values = raw_valid.loc[first_index, selected]
terms = pd.DataFrame({"Значення ознаки": first_values, "Коефіцієнт": beta_raw,
                      "Добуток": first_values * beta_raw})
z_first = intercept_raw + terms["Добуток"].sum()
print("Ймовірність за формулою:", expit(z_first))
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

Order	SalePrice	Фактичний клас	P(Y=1)	Клас за порогом 0.5
2925	131000	1	0.9995753	1
2360	138800	1	0.69015745	1
433	610000	1	1	1
1717	204000	1	0.995987	1
765	159900	1	0.52943987	1

	Значення ознаки	Коефіцієнт	Добуток
LotFrontage	160	0.020470732	3.2753171
LotArea	20000	7.9491266e-05	1.5898253
OverallQual	5	0.81739431	4.0869716
OverallCond	7	0.74800588	5.2360412
YearBuilt	1960	0.04919673	96.42559
YearRemodAdd	1996	0.017700964	35.331123
BsmtFinSF1	1224	0.002641284	3.2329316
BsmtFinSF2	0	0.0023383025	0
BsmtUnfSF	0	0.0019714091	0
1stFlrSF	1224	0.004114672	5.0363585
2ndFlrSF	0	0.0042424299	0
BsmtFullBath	1	0.89737154	0.89737154
BsmtHalfBath	0	0.5101582	0
FullBath	1	0.6857339	0.6857339
HalfBath	0	0.47773137	0
KitchenAbvGr	1	-2.3075442	-2.3075442
Fireplaces	1	0.61245528	0.61245528
GarageCars	2	0.43007773	0.86015545
GarageArea	576	0.0014827435	0.85406024
WoodDeckSF	474	0.0021723264	1.0296827
EnclosedPorch	0	0.0025260833	0
ScreenPorch	0	0.0085011718	0
PoolArea	0	-0.005498648	-0
YrSold	2006	-0.10616134	-212.95964

```
Константа: 63.877265241644466
z першого будинку: 7.763698292506341
Ймовірність за формулою: 0.9995752981160083
```

Повна підстановка для першого будинку (порядок ознак як у таблиці; показані числа округлено):

$$
\begin{aligned}
z \approx\ & 63.8772652416 \\
& + 0.020470732147 \cdot 160 \\
& + 0.000079491266 \cdot 20000 \\
& + 0.817394310330 \cdot 5 \\
& + 0.748005880712 \cdot 7 \\
& + 0.049196729518 \cdot 1960 \\
& + 0.017700963663 \cdot 1996 \\
& + 0.002641283975 \cdot 1224 \\
& + 0.002338302470 \cdot 0 \\
& + 0.001971409081 \cdot 0 \\
& + 0.004114671961 \cdot 1224 \\
& + 0.004242429923 \cdot 0 \\
& + 0.897371535405 \cdot 1 \\
& + 0.510158195409 \cdot 0 \\
& + 0.685733903603 \cdot 1 \\
& + 0.477731367771 \cdot 0 \\
& - 2.307544158146 \cdot 1 \\
& + 0.612455279815 \cdot 1 \\
& + 0.430077726125 \cdot 2 \\
& + 0.001482743473 \cdot 576 \\
& + 0.002172326428 \cdot 474 \\
& + 0.002526083256 \cdot 0 \\
& + 0.008501171822 \cdot 0 \\
& - 0.005498647983 \cdot 0 \\
& - 0.106161336239 \cdot 2006 \\
& \approx 7.76369829251
\end{aligned}
$$

$$
p = \frac{1}{1 + e^{-(7.76369829251)}} \approx 0.999575298116.
$$

Результати та інтерпретація

За стандартного порогу прогнозуємо клас 1, якщо $p \geq 0.5$, і клас 0 інакше. Тут p є прогнозованою ймовірністю ціни понад 130000 доларів.

У виводі наведено ймовірності `.predict()` для п'яти випадково вибраних будинків і повну числову підстановку для першого. Для будинку `Order=2925` показник z, тобто сума вільного члена й добутків коефіцієнтів на значення ознак, становить приблизно 7.7636983, а ймовірність 0.9995753, тому прогнозуємо клас 1. Фактична ціна цього будинку становить 131000 доларів. Усі п'ять випадково вибраних будинків цього разу належать до класу 1.

3.д Матриця помилок при 0.5

Умова завдання

Побудуйте матрицю помилок за допомогою функції confusion_matrix з sklearn.metrics для стандартного порогу 0.5. Розрахуйте чутливість (recall / sensitivity), специфічність та загальну точність (accuracy). Надайте розгорнуту словесну інтерпретацію кожної клітинки матриці помилок у контексті вартості та доступності будинків.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
valid_probabilities = model.predict(Z_valid[["const"] + selected])
def threshold_metrics(threshold):
    predicted = (valid_probabilities >= threshold).astype(int)
    matrix = confusion_matrix(y_valid, predicted, labels=[0, 1])
    tn, fp, fn, tp = matrix.ravel()
    metrics = {"Поріг": threshold, "TN": tn, "FP": fp, "FN": fn, "TP": tp,
               "Чутливість": tp / (tp + fn), "Специфічність": tn / (tn + fp),
               "Accuracy": (tp + tn) / matrix.sum(), "Прогнозів класу 1": predicted.sum()}
    return matrix, metrics
matrix_default, metrics_default = threshold_metrics(0.5)
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

	Прогноз 0	Прогноз 1
Факт 0	125	23
Факт 1	19	419

	Значення
Поріг	0.5
TN	125
FP	23
FN	19
TP	419
Чутливість	0.956621
Специфічність	0.84459459
Accuracy	0.92832765
Прогнозів класу 1	442

```
Accuracy постійного прогнозу класу 1: 0.7474402730375427
```

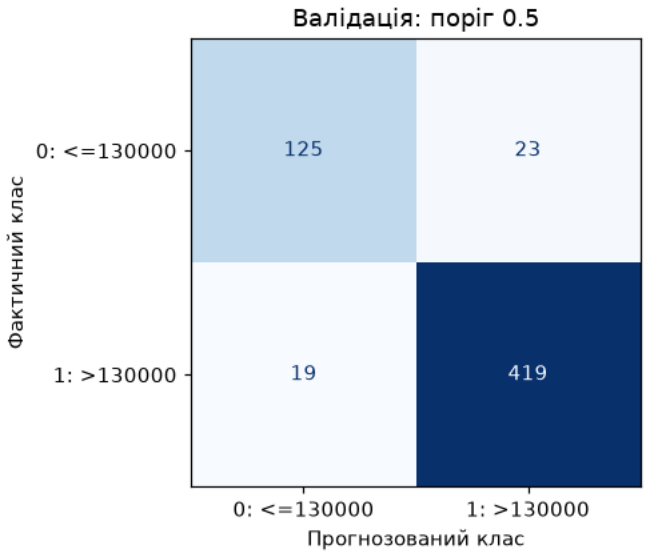


Рис. 7: Матриця помилок при порозі 0.5.

Результати та інтерпретація

У матриці рядки відповідають фактичному класу, а стовпці прогнозованому. Чутливість дорівнює $TP/(TP+FN)$, специфічність $TN/(TN+FP)$, а загальна точність $(TP+TN)/N$. Тут TP і TN є кількостями правильних прогнозів класів 1 і 0, FP і FN кількостями хибних прогнозів цих класів, а N кількістю валідаційних будинків.

- TN (125). Фактична ціна вкладається в бюджет 130000 доларів, і модель правильно визначила ці будинки як доступні за цим порогом.
- FP (23). Фактична ціна вкладається в бюджет, але модель спрогнозувала дорожчу групу. Покупець із таким бюджетом може помилково відкинути ці будинки.
- FN (19). Фактична ціна перевищує бюджет, але модель спрогнозувала доступну групу. Покупець може помилково очікувати, що ці будинки вкладаються в бюджет.
- TP (419). Фактична ціна перевищує бюджет, і модель правильно віднесла ці будинки до дорожчої групи.

За порогу 0.5 чутливість становить 0.9566, специфічність 0.8446, а загальна точність 0.9283.

3.e ROC та поріг Юдена

Умова завдання

Оскільки поріг 0.5 не завжди є оптимальним, побудуйте ROC-криву за допомогою roc_curve та знайдіть площу під нею (roc_auc_score). Визначте оптимальний поріг класифікації, максимізувавши індекс Юдена (Sensitivity+Specificity−1). Побудуйте графік кривої за допомогою matplotlib.

Код Python

```python
# Вище: імпорти, налаштування та підготовка змінних.
fpr, tpr, thresholds = roc_curve(y_valid, valid_probabilities, drop_intermediate=False)
auc_value = roc_auc_score(y_valid, valid_probabilities)
youden = tpr - fpr
finite = np.flatnonzero(np.isfinite(thresholds))
# При однаковому максимумі беремо найбільший поріг: roc_curve повертає їх за спаданням.
best = finite[np.argmax(youden[finite])]
optimal_threshold = thresholds[best]
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

	Значення
ROC AUC	0.97280328
Поріг Юдена	0.67279019
Індекс Юдена	0.84129335
Чутливість	0.92237443
Специфічність	0.91891892

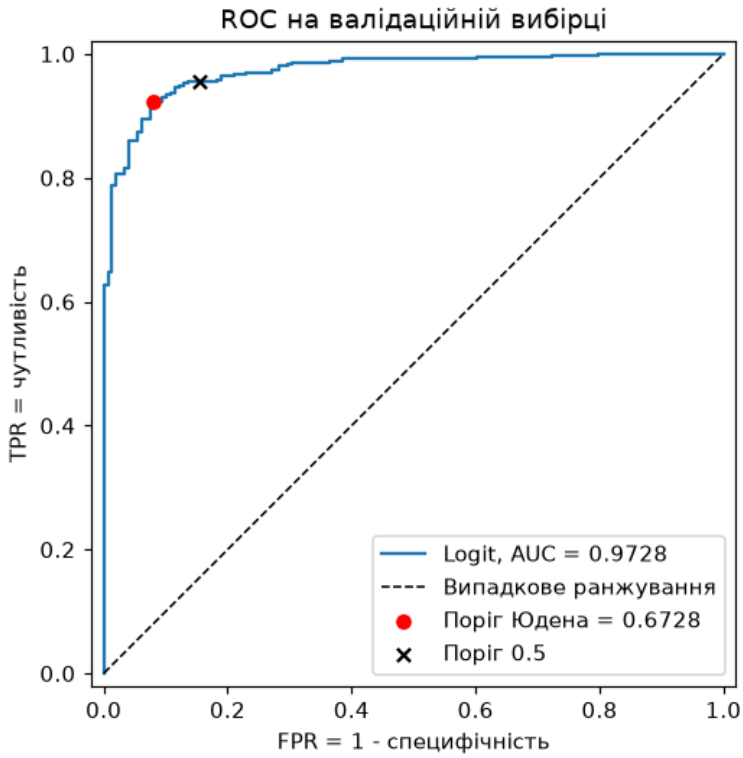


Рис. 8: ROC-крива й обраний поріг Юдена.

Результати та інтерпретація

ROC показує, як чутливість і частка хибнопозитивних прогнозів змінюються з порогом. AUC є площею під цією кривою та характеризує, наскільки добре модель ранжує будинки двох класів.

Вибираємо поріг, що максимізує $J = \text{TPR} - \text{FPR}$. Тут J є індексом Юдена, TPR чутливістю, а FPR часткою хибнопозитивних прогнозів серед будинків класу 0. Оскільки FPR дорівнює одиниці мінус специфічність, цей критерій дає однакову вагу чутливості та специфічності.

AUC на валідаційній вибірці становить 0.9728. Найбільший індекс Юдена 0.8413 отримали за порогу 0.6728. Оскільки поріг вибрано на цій самій вибірці, метрики за ним описують результат на даних, використаних для вибору порогу.

3.ж Класифікація з новим порогом

Умова завдання

Перерахуйте матрицю помилок, чутливість та специфічність для нового (оптимального за ROC) порогу. Порівняйте ці метрики з отриманими при порозі 0.5. Дайте відповідь: чи стала ваша модель після зміни порогу більш «консервативною» щодо прогнозування дорогих будинків (клас 1)?

Код Python

```
# Вище: імпорти, налаштування та підготовка змінних.
matrix_optimal, metrics_optimal = threshold_metrics(optimal_threshold)
comparison = pd.DataFrame([metrics_default, metrics_optimal], index=["Стандартний", "За Юденом"])
# Далі: таблиці, графіки й перевірки; фактичні виводи наведено нижче.
```

	Стандартний	За Юденом
Поріг	0.5	0.67279019
TN	125	136
FP	23	12
FN	19	34
TP	419	404
Чутливість	0.956621	0.92237443
Специфічність	0.84459459	0.91891892
Accuracy	0.92832765	0.92150171
Прогнозів класу 1	442	416

	Прогноз 0	Прогноз 1
Факт 0	136	12
Факт 1	34	404

```
Зміна кількості прогнозів класу 1: -26
Поріг підвищився: True
```

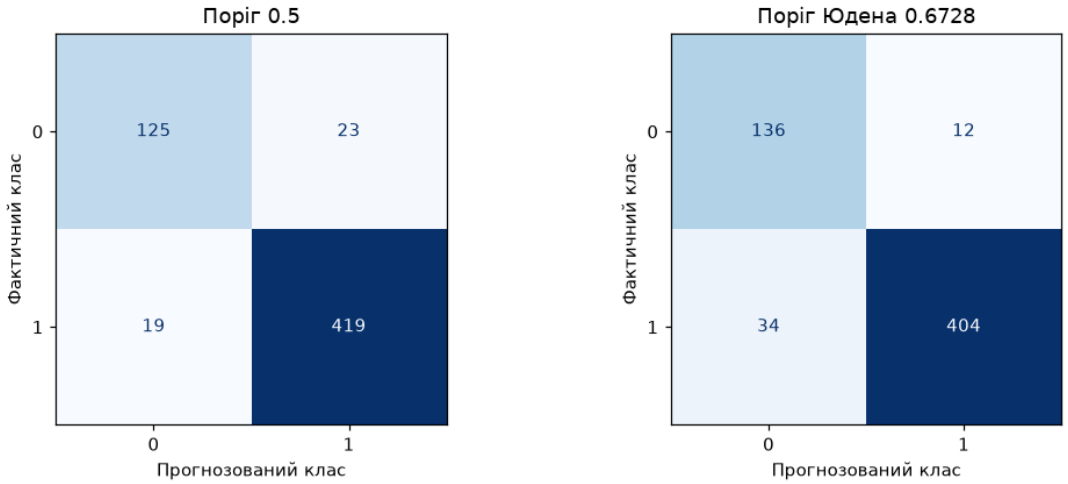


Рис. 9: Порівняння матриць помилок для двох порогів.

Результати та інтерпретація

Змінюємо лише поріг перетворення ймовірності на клас. За вищого порогу моделі потрібна більша прогнозована ймовірність, щоб віднести будинок до класу 1.

Правило стало більш консервативним щодо дорогих будинків. Поріг зріс із 0.5 до 0.6728, і кількість прогнозів класу 1 зменшилася з 442 до 416. Хибнопозитивних прогнозів стало 12 замість 23, а хибнонегативних 34 замість 19. Специфічність зросла з 0.8446 до 0.9189, а чутливість зменшилася з 0.9566 до 0.9224. Модель рідше помилково називає доступний будинок дорогим, але частіше пропускає справді дорогі будинки.